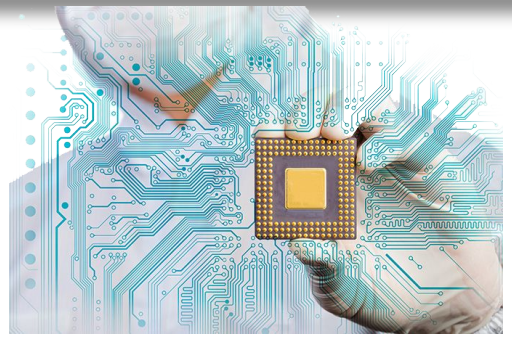
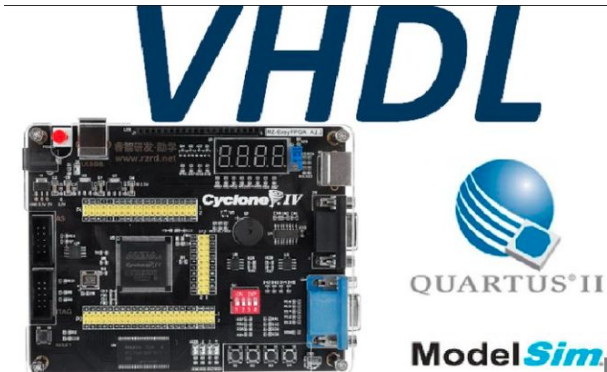
UNIDADES BÁSICAS DE DISEÑO EN VHDL

PRINCIPIO DE DISEÑO VLSI
DISEÑO DIGITAL CON TÉCNICAS DE ALTO NIVEL
ÁREA DISPOSITIVOS
DEPTO. DE ELECTRÓNICA Y COMPUTACIÓN
FI.UNMdP



UNIDADES BÁSICAS DE DISEÑO EN VHDL

- UNIDADES DE DISEÑO EN VHDL
- ENTIDAD
- ARQUITECTURA
- CONFIGURACIÓN
- PAQUETES
- SECUENCIA DE COMPILACIÓN



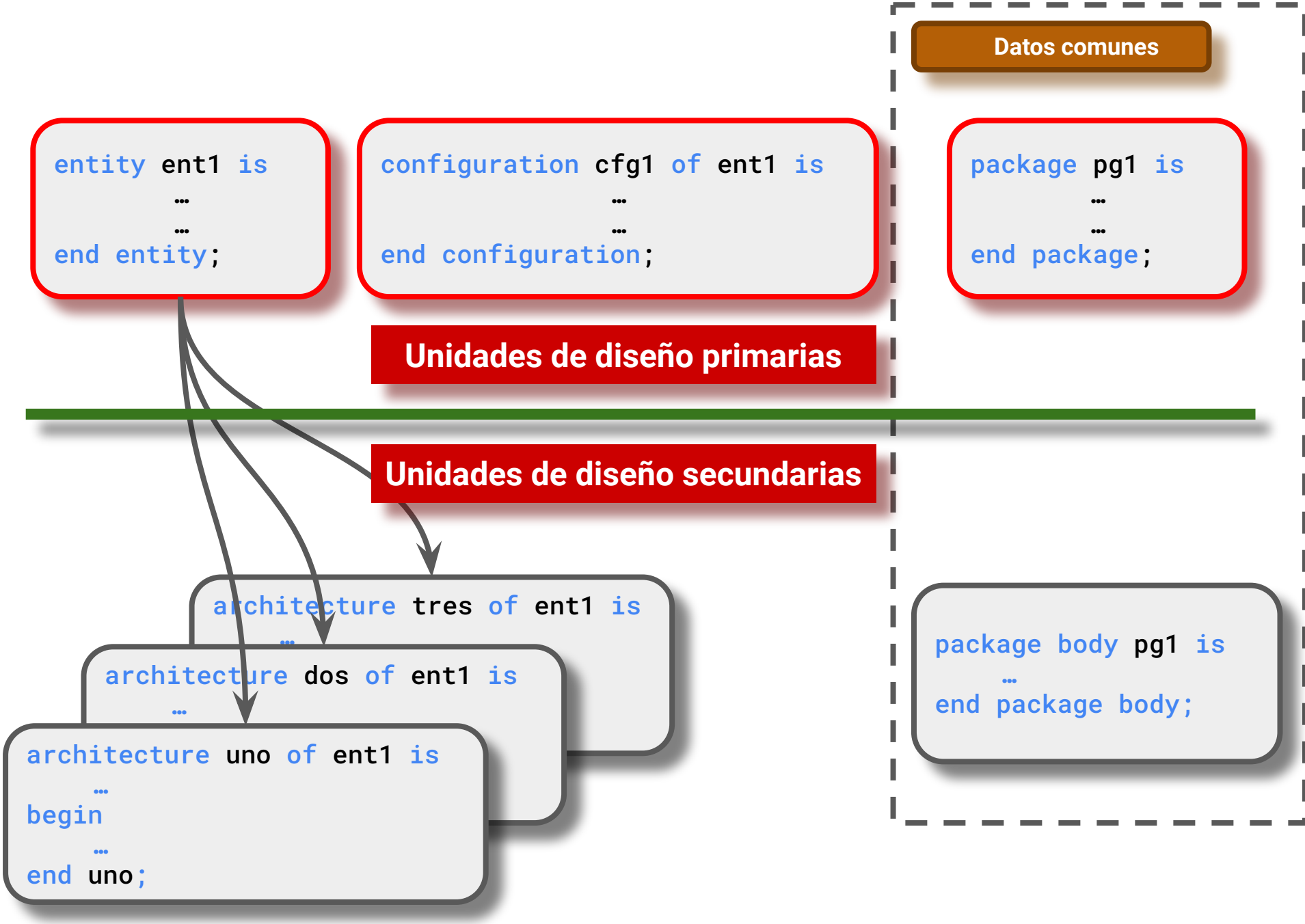
TIPO DE UNIDADES DE DISEÑO

- DE ENTIDAD (ENTITY DECLARATION)
- DE ARQUITECTURA (ARCHITECTURE)
- DE CONFIGURACIÓN (CONFIGURATION)
- DECLARACIÓN DE PAQUETE
- CUERPO DEL PAQUETE

VHDL

Entidad, declaración de paquete y configuración => se denominan unidades primarias

Arquitectura y cuerpo de paquete => unidades secundarias, dependen de la unidad primaria, que se debe analizar antes que estas


Datos comunes
entity ent1 is
…
…
end entity;
configuration cfg1 of ent1 is
…
…
end configuration;
package pg1 is
…
…
end package;
Unidades de diseño primarias
Unidades de diseño secundarias
architecture tres of ent1 is
…
architecture dos of ent1 is
…
architecture uno of ent1 is
…
begin
…
end uno;
package body pg1 is
…
end package body;

• ENTIDAD

Sirve para definir los puertos de un circuito

Es la interfaz entre un módulo y su entorno

No da información sobre su realización

SINTAXIS:

```
entity <entity_name> is
   generic
   (
      <name>   : <type>  :=   <default_value>;
      <name>   : <type>  :=   <default_value>
   );
   port
   (
      -- Input ports
      <name>   : in  <type>;
      <name>   : in  <type> := <default_value>;
      -- Inout ports
      <name>   : inout <type>;
      -- Output ports
      <name>   : out <type>;
      <name>   : out <type> := <default_value>
   );
end <entity_name>;
```

- ENTIDAD

PUERTOS DE UNA ENTIDAD

Sirven para indicar los puntos de comunicación con el mundo exterior

Cada puerto tiene:

- NOMBRE
- TIPO
- MODO

EJEMPLO

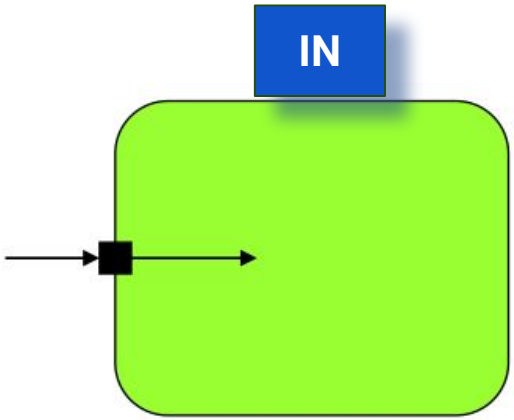
```
entity mux21 is
   port( a,b : in  std_logic;
         sel : in  std_logic;
           c : out std_logic
       );
end entity;
```

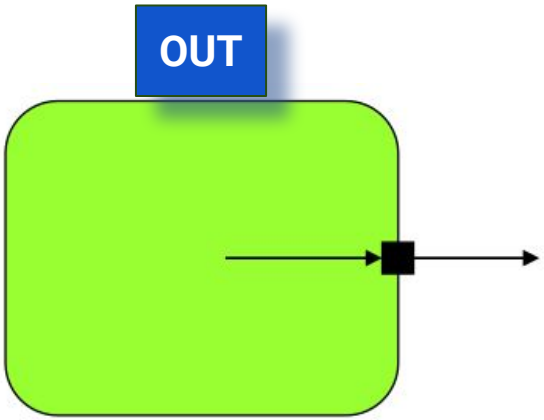
Opcionalmente se les puede asignar un valor por omisión, que se emplea para la simulación y sólo cuando el puerto está desconectado.

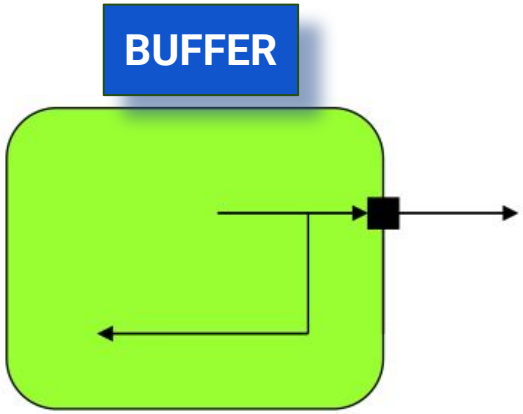
- ENTIDAD

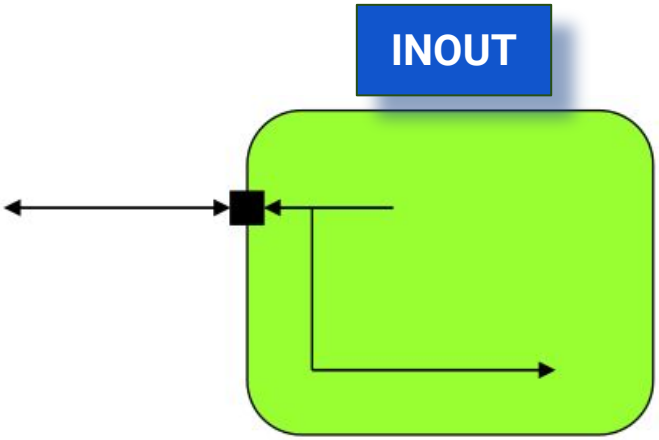
PUERTOS DE UNA ENTIDAD

- MODOS









- **ENTIDAD**

GENÉRICOS (generic)

Genéricos: conjunto de parámetros que permiten cambiar la funcionalidad del circuito.

EJEMPLO

```
entity mux21 is

   generic (N : integer := 2);

   port( a,b : in  std_logic_vector(N-1 downto 0);
         sel : in  std_logic;
           c : out std_logic_vector(N-1 downto 0)
       );

end entity;
```

• ARQUITECTURA

Se define la funcionalidad de la entidad que representa

SINTAXIS:

```
architecture <arch_name> of <entity_name> is

   -- Declarations (optional)

begin

   -- Process Statement (optional)

   -- Concurrent Procedure Call (optional)

   -- Concurrent Signal Assignment (optional)

   -- Conditional Signal Assignment (optional)

   -- Selected Signal Assignment (optional)

   -- Component Instantiation Statement (optional)

   -- Generate Statement (optional)

end <arch_name>;
```

La parte de declaraciones está destinada para declarar los elementos necesarios para la descripción (constantes,tipos de datos, señales internas, etc.)

La sección de sentencias concurrentes sirve para describir la funcionalidad del dispositivo

• ARQUITECTURA

ESTILOS

Según el tipo de sentencias la arquitectura se puede modelar de forma:

- **Algorítmico:** se define la funcionalidad del dispositivo mediante un conjunto de instrucciones que se ejecutan secuencialmente
- **Flujo de datos:** modela la arquitectura como un flujo de datos entre distintos módulos encargados de implementar funciones u operadores
- **Estilo estructural:** define la arquitectura como un conjunto de componentes interconectados
- **Estilo mixto:** combinaciones de los otros estilos

• ARQUITECTURA

ESTILOS

ALGORÍTMICO (COMPORTAMENTAL)

Se define la funcionalidad del dispositivo mediante un algoritmo en el que, como en cualquier lenguaje de alto nivel, no hay ninguna referencia al hardware

```
architecture algoritmico of mux21 is

begin

   process(a,b,sel)
   begin
      if (sel = '0') then
         c <= a;
      else
         c <= b;
      end if;
   end process;

end algoritmico;
```

• ARQUITECTURA

ESTILOS

FLUJO DE DATOS

Se describe la funcionalidad mediante ecuaciones que se ejecutan de manera concurrente

```
architecture flujodatos of mux21 is

   signal sel_n, n1, n2 : std_logic;

begin

   sel_n <= not(sel) after 1 ns;
   n1 <= sel_n and a after 2 ns;
   n2 <= sel and b after 2 ns;
   c <= (n1 or n2) after 2 ns;

end flujodatos;
```

• ARQUITECTURA

ESTILOS

ESTRUCTURAL

Se trata de un conjunto de componentes interconectados mediante señales. La descripción es puramente estructural, se parece a un listado de red o netlist

```
architecture estructural of mux21 is

   signal sel_n,n1,n2 : std_logic;

   component inv
      port ( y : in std_logic;
      z : out std_logic );
   end component;

   component and2
      port ( x : in std_logic;
      y : in std_logic;
      z : out std_logic );
   end component;

   component or2
      port ( x : in std_logic;
      y : in std_logic;
      z : out std_logic );
   end component;
```

```
begin

   u0: inv port map(sel, sel_n);
   u1: and2 port map(sel_n,a,n1);
   u2: and2 port map(sel,b,n2);
   u3: or2 port map(n1,n2,c);

end estructural;
```

- ARQUITECTURA

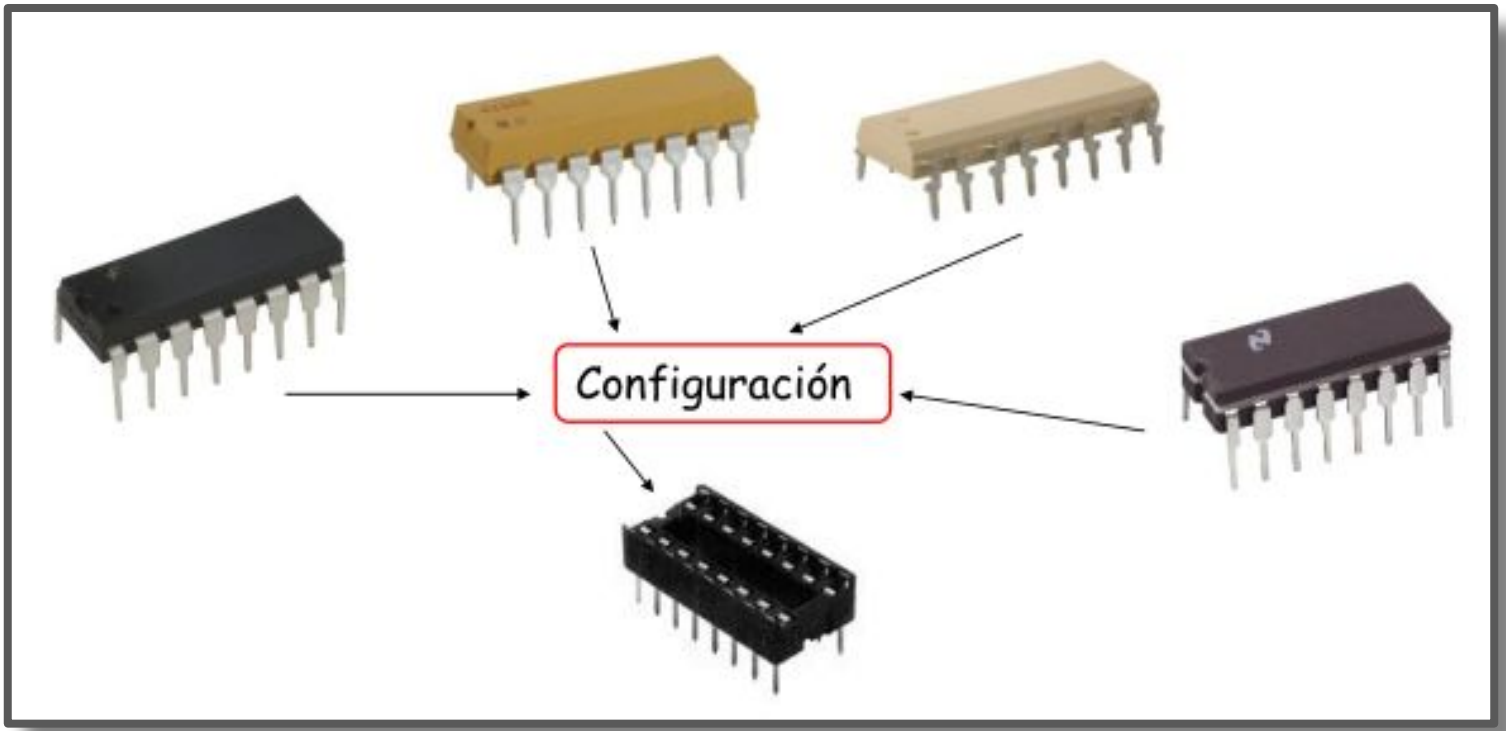
ESTILOS

MIXTO

Una arquitectura puede, y de hecho a menudo lo está, compuesta por una combinación de los tres estilos. VHDL brinda un grado muy alto de flexibilidad.

• CONFIGURACIÓN

La configuración es una unidad de diseño VHDL. Determina la arquitectura a emplear para una entidad determinada

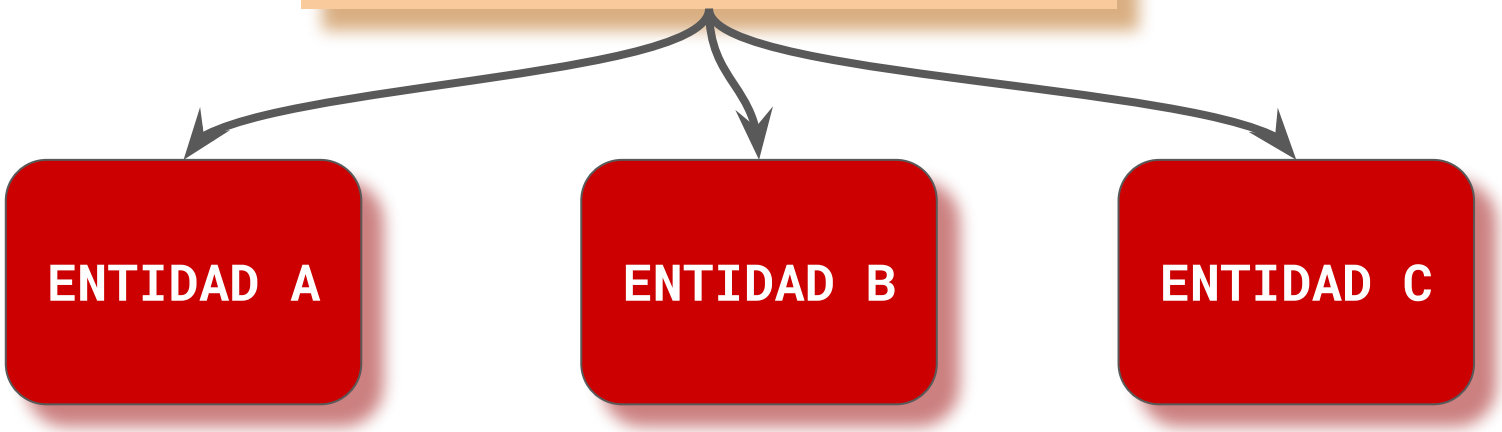


• PAQUETES

Colección de datos comunes

```
package project_pack is
    -- constantes
    -- tipos de datos
    -- componentes
    -- sub rutinas
    -- …
end project_pack;
```

```
Use work.project_pack.all;
```



• PAQUETES

El paquete se divide en dos unidades bien diferenciadas

Declaración de paquete: es la visión externa de los elementos que se declaran. Suelen declararse subprogramas dándoles un nombre y una lista de parámetros pero sin incluir su cuerpo. Se pueden incluir constantes y asignarles su valor aquí o en el cuerpo.

```
package identificador
   [declaraciones]
end package identificador;
```

Cuerpo del paquete: se declara la implementación de los elementos

```
package body identificador is
   [declaraciones cuerpo]
end package body identificador;
```

• PAQUETES

EJEMPLO

Paquete para definir el retardo de los operadores lógicos not, and y or (se puede incluir toda la información en la declaración del paquete, en este ejemplo se utiliza el cuerpo para dar valor a las constantes)

```
package retardosop is
   constant retNOT : time;
   constant retaAND2 : time;
   constant retOR2 : time;
end retardosop;

package body retardosop is
   constant retNOT : time := 1 ns;
   constant retaAND2 : time := 2 ns;
   constant retOR2 : time := 2 ns;
end retardosop;
```

El resultado del análisis de un paquete se almacena en una biblioteca.
Se lo puede utilizar:

- Para un único elemento del paquete: use biblio.retardosop.retNOT;
- Para referenciar todos los elementos del paquete: use biblio.retardosop.all;

• PAQUETES

EJEMPLO

Para el ejemplo mux21 se puede emplear el paquete anterior

```
use biblio.retardosop.all;

architecture flujodatos of mux21 is

   signal sel_n, n1, n2 : std_logic;

begin

   sel_n <= not(sel) after retNOT;
   n1 <= sel_n and a after retAND2;
   n2 <= sel and b after retAND2;
   c <= (n1 or n2) after retOR2;

end flujodatos;
```

• SECUENCIA DE COMPILACIÓN

Las unidades primarias se analizan primero
- La entidad antes que la arquitectura
- La declaración de un paquete antes que su cuerpo

Todas las unidades que son referenciadas se analizan primero
- Entidad después que los paquetes
- La configuración después de las entidades y arquitecturas